

Autopoiesis: A Self-Evolving System Paradigm for LLM Serving Under Runtime Dynamics

发表：2026/04/08

管理收藏夹

TL;DR 与摘要

思维导图

论文图片分析

相似文献推荐

PDF

原文

精读

论文状态：失败

内容目录

展开目录 ▾

1. 论文基本信息

1.1. 标题

论文标题为 **Autopoiesis: A Self-Evolving System Paradigm for LLM Serving Under Runtime Dynamics**，核心主题是面向运行时动态性的大语言模型服务自演化系统范式，提出了一种能够在线自动优化服务策略的LLM服务架构。

1.2. 作者与隶属机构

作者团队及所属机构如下：

- Youhe Jiang*、Ran Yan*、You Peng、Wenshuang Li、Binhang Yuan †：香港科技大学 (HKUST)
- Taiyi Wang：Powersense Technology Limited
- Fangcheng Fu †：上海交通大学 其中*表示同等贡献，†表示通讯作者，研究团队深耕LLM服务系统、分布式系统、程序合成等领域。

1.3. 发表状态与来源

该论文为**预印本**，2026年4月8日发布于arXiv，尚未正式发表在期刊或会议上。



1.4. 摘要

本研究针对现有LLM服务依赖人工设计的静态策略、无法适配运行时动态波动（负载波动、集群弹性扩缩、spot实例抢占等）的痛点，提出了名为Autopoiesis的自演化LLM服务范式：

1. 采用双平面架构，将策略执行与策略演化解耦，数据面负责处理线上请求，控制面负责异步优化服务策略；
2. 提出LLM驱动的程序合成工作流，结合演化算法、运行时反馈自动生成适配当前环境的服务策略代码，平衡调度开销、重配置开销、服务效率之间的复杂权衡；
3. 实现策略在线热替换，无需中断线上服务即可完成策略迭代。实验结果表明，Autopoiesis在多种动态场景下相比现有SOTA LLM服务系统最高取得53%的性能提升，平均提升34%。

1.5. 原文链接

- 预印本页面：<https://arxiv.org/abs/2604.07144>
- PDF链接：<https://arxiv.org/pdf/2604.07144v1>

2. 整体概括

2.1. 研究背景与动机

2.1.1. 核心问题

LLM服务的生产环境存在大量不可预测的动态性：

- 工作负载动态：用户请求的到达率、prompt长度、生成长度随时波动，存在流量突增、长文本请求占比变化等情况；
- 集群资源动态：云环境中弹性扩缩容会增减计算节点，spot实例可能被随机抢占，硬件故障也会导致节点离线。现有LLM服务系统依赖**人工设计的静态服务策略**（包含调度算法、重调度策略），需要平衡多组深度耦合的权衡：
 1. 重调度频率vs重配置开销：频繁重调度能保证策略适配最新环境，但会带来更多的模型迁移、KV缓存迁移开销；
 2. 调度 thoroughness（调度深度）vs执行效率：调度时投入更多计算资源能生成更优的部署计划，但会延长调度时间，期间系统只能用过时的旧计划服务，导致性能下降；
 3. 重配置激进程度vs服务稳定性：完全重配置到全局最优计划能最大化后续服务效率，但重配置过程中的性能损失更大。这些权衡的最优平衡点是**场景专属、随运行时动态变化的**，静态策略

无法适配所有场景和动态变化，导致性能下降。

2.1.2. 现有研究的空白

现有LLM服务系统（如Llumnix、DistServe、SpotServe等）的策略都是人工预先设计的静态规则，存在两个核心局限：

1. 人工很难穷尽所有场景的最优权衡，尤其是多维度耦合的复杂权衡空间；
2. 静态策略部署后不会变化，一旦运行时环境偏离调优场景，性能就会大幅下降。而现有LLM驱动的程序合成工作都是离线生成静态代码，无法适配动态变化的运行时环境，也没有应用到LLM服务策略优化的场景。

2.1.3. 创新思路

本文将LLM服务从「静态人工策略部署」转变为「连续在线策略演化」：用LLM驱动的程序合成技术，结合实时运行时反馈，自动在线生成适配当前环境的服务策略代码，无需人工干预，且策略更新不中断线上服务。

2.2. 核心贡献与主要发现

论文的核心贡献分为四点：

1. **LLM驱动的服务策略合成工作流**：结合演化算法、轻量模拟器评估、结构化反馈引导，自动探索多维度权衡空间，生成适配当前运行时的最优服务策略；
2. **自演化服务系统架构**：设计了数据面-控制面解耦的双平面架构，数据面执行当前策略处理请求，控制面异步完成策略演化和热替换，不影响线上服务；
3. **全场景性能验证**：在同构集群、异构集群、弹性云集群三种动态场景下的实验表明，Autopoiesis相比SOTA系统最高提升53%，平均提升34%；
4. **系统分析**：验证了演化策略能自适应不同场景的权衡需求，且该合成范式可以推广到更多系统优化问题。

3. 预备知识与相关工作

3.1. 基础概念

为了帮助初学者理解，首先对本文涉及的核心术语进行解释：

3.1.1. LLM服务 (LLM Serving)

指将训练好的大语言模型部署为在线服务，接收用户的文本请求 (prompt)，执行推理生成返回结果的完整过程。生产级LLM服务需要处理高并发请求、保证低延迟、高吞吐量，同时最大化硬件利用率。

3.1.2. 服务策略 (Serving Policy)

是LLM服务系统的核心决策逻辑，包含两个核心组件：

- **调度算法 (Scheduling Algorithm)**：负责生成服务计划，决定请求怎么分配到GPU、模型怎么部署到GPU、采用什么并行策略 (张量并行、流水线并行等)，目标是最大化服务效率；
- **重调度策略 (Rescheduling Strategy)**：负责决定什么时候触发重调度 (比如负载变化超过阈值、节点离线时)，以及重配置的激进程度，目标是平衡重配置开销和收益。

3.1.3. LLM驱动的程序合成 (LLM-driven Program Synthesis)

是一种新兴的代码生成范式，结合LLM的代码生成能力和演化算法的迭代优化能力，自动探索巨大的代码设计空间，生成比人工编写更优的程序/算法。

3.1.4. 重配置开销 (Reconfiguration Overhead)

指调整服务部署计划时产生的性能损失，包括模型权重迁移、KV缓存迁移、并行策略调整期间的服务效率下降等。

3.2. 前人工作

3.2.1. LLM服务系统

近年来的LLM服务系统主要针对特定场景优化静态策略：

- **DistServe**：将prefill (预填充) 阶段和decode (解码) 阶段拆分到不同GPU执行，解决两类请求的干扰问题，优化工作负载波动场景的性能；
- **HexGen**：针对异构GPU集群设计调度策略，适配不同型号GPU的算力、显存差异；
- **SpotServe**：针对抢占式实例 (spot instance) 场景，设计状态恢复、动态重配置策略，降低实例抢占带来的性能损失。这些系统的策略都是人工预先设计的静态规则，无法随运行时动态调整权衡平衡点。

3.2.2. LLM驱动的程序合成

现有LLM程序合成工作已经在多个领域取得突破：

- AlphaEvolve：结合LLM和演化算法，自动发现新的数学优化算法；
- KernelEvolve：自动生成优化的GPU核函数，性能超过人工编写的版本。但这些工作都是离线生成静态代码，针对固定场景优化，无法适配动态变化的运行时环境，也没有应用到LLM服务策略优化领域。

3.3. 技术演进与差异化分析

LLM服务策略的演进经历了三个阶段：

1. 固定规则策略：人工编写简单的静态规则（如固定间隔重调度、贪心分配请求），适配性差；
2. 自适应规则策略：人工设计带阈值的自适应规则（如负载变化超过X%时触发重调度），但阈值需要人工调优，只能覆盖有限场景；
3. 自演化策略（本文提出）：无需人工设计规则，由LLM在线自动生成适配当前环境的策略，随运行时动态迭代。本文与现有工作的核心差异是：首次将LLM驱动的在线程序合成引入LLM服务领域，把静态人工策略变成了随运行时动态演化的「活代码」，解决了静态策略无法适配动态环境的痛点。

4. 方法论

4.1. 方法原理

Autopoiesis的核心思想是**策略执行与策略演化解耦**：用数据面负责低延迟处理线上请求，用异步运行的控制面负责根据实时运行时数据演化更优的服务策略，演化完成后通过热替换机制更新数据面的策略，整个过程不需要中断线上服务。其背后的直觉是：运行时动态变化是连续、渐变的，上一时刻的最优策略可以作为下一时刻演化的起点，通过小步迭代的方式持续优化，不需要每次都从零开始设计策略。

4.2. 核心方法详解

4.2.1. 问题形式化

首先定义服务策略的优化目标和约束：

优化目标

给定运行时快照（包含最近的负载、集群状态数据），目标是演化出最优的服务策略，最小化端到端trace完成时间 T_{total} ，公式如下（严格忠于原文）： $\mathrm{T}_{\mathrm{total}} = \mathrm{t}_{\mathrm{sched}}(1) + \mathrm{t}_{\mathrm{serve}}(1) + \sum_{i=2}^N [\mathrm{t}_{\mathrm{stale}}(i) + \mathrm{t}_{\mathrm{reconfig}}(i) + \mathrm{t}_{\mathrm{serve}}(i)]$ 符号解释：

- $t_{sched}(1)$ ：冷启动阶段第一次生成初始服务计划的调度开销；
- $t_{serve}(1)$ ：第一次调度完成后，用初始计划服务的时间；
- N ：整个trace周期内重调度的总次数，由重调度策略动态决定；
- $t_{stale}(i)$ ：第 i 次重调度期间，系统用旧的过时计划服务的时间开销（调度阶段服务效率下降）；
- $t_{reconfig}(i)$ ：第 i 次重配置阶段，系统切换到新计划期间的的时间开销（重配置阶段服务效率下降）；
- $t_{serve}(i)$ ：第 i 次重配置完成后，用新计划服务的时间。

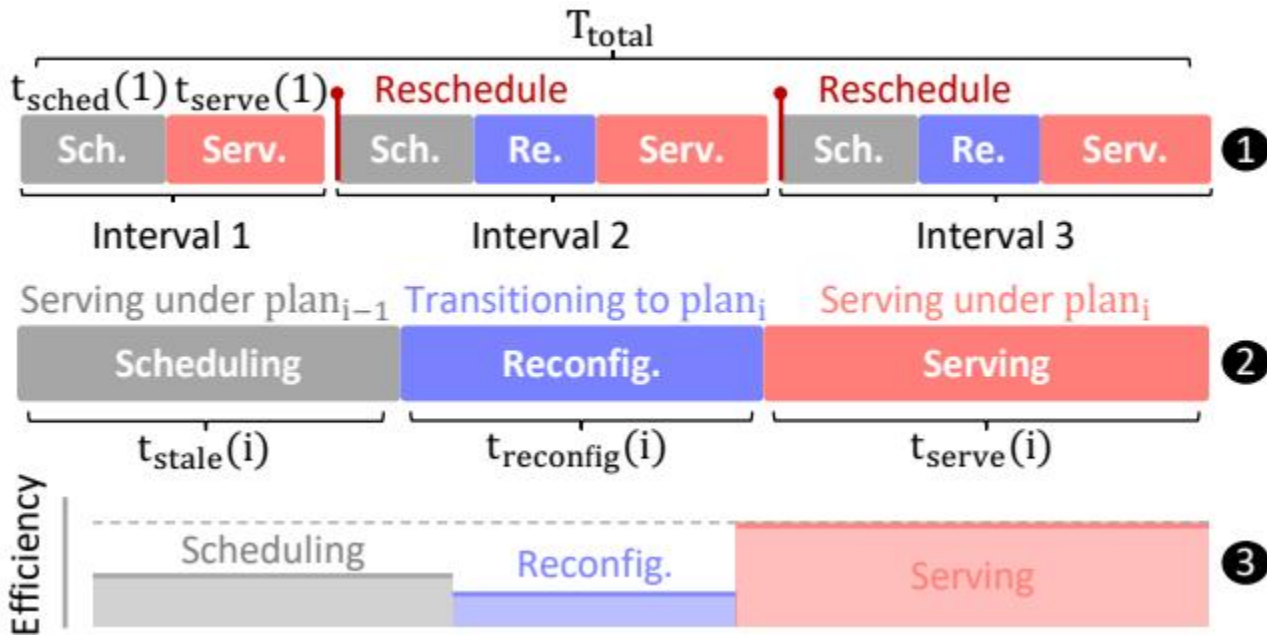
服务策略定义

服务策略由两个核心函数组成：

1. $\text{should_reschedule}(ctx) \rightarrow \{0, 1\}$ ：重调度策略，输入当前运行时上下文 ctx （包含负载、集群资源、当前部署计划等状态），输出是否需要触发重调度，决定了重调度频率 N ；
2. $\text{schedule}(ctx) \rightarrow plan$ ：调度算法，输入当前上下文，输出新的服务计划（包含工作负载分配、资源分配、并行策略），决定了调度开销、重配置开销和后续服务效率。

连续执行约束

LLM服务是连续在线的，调度和重配置期间服务不会中断，因此调度和重配置的开销会直接影响整体性能，这也是三组核心权衡的来源。执行模型的三个阶段如下图（原文Figure4）所示：

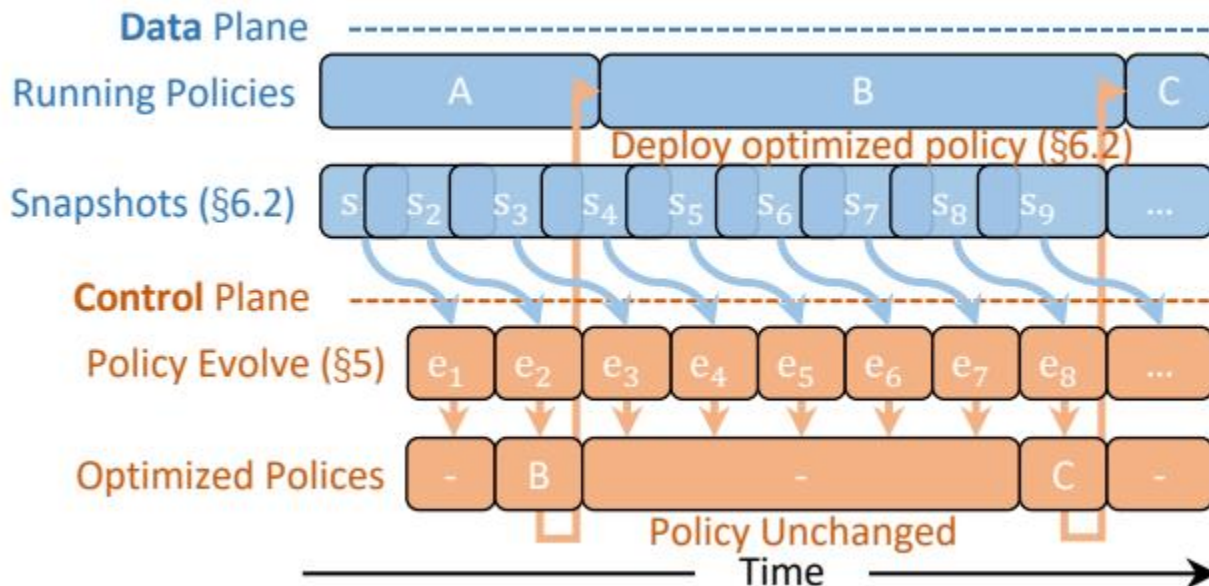


该图像是示意图，展示了执行模型及其在总时间 T_{total} 内的调度和重配置过程。图中标记①显示了由冷启动和重调度触发的调度间隔；②则详细描述了第二个及之后的间隔，包括调度、重配置和服务三个阶段；③展示了运行效率的变化情况，调度和重配置期间效率下降，而服务期间效率达到最高。

- 阶段1（调度）：触发重调度后，调度算法生成新计划，同时系统继续用旧计划服务，效率较低，对应开销 $t_{stale}(i)$ ；
- 阶段2（重配置）：系统从旧计划切换到新计划，部分资源用于迁移模型、KV缓存，服务效率下降，对应开销 $t_{reconfig}(i)$ ；
- 阶段3（服务）：新计划生效，系统以最高效率服务，对应开销 $t_{serve}(i)$ 。

4.2.2. 双平面系统架构

Autopoiesis采用数据面-控制面解耦的双平面架构，如下图（原文Figure3）所示：



该图像是一个示意图，展示了数据平面和控制平面之间的互动关系，以及在运行政策和优化政策之间的演变过程。图中显示了运行政策 A、B、C，快照和时间轴，及政策演变的步骤 $e_1, e_2, ext..., e_n$ 。

数据面（策略执行层）

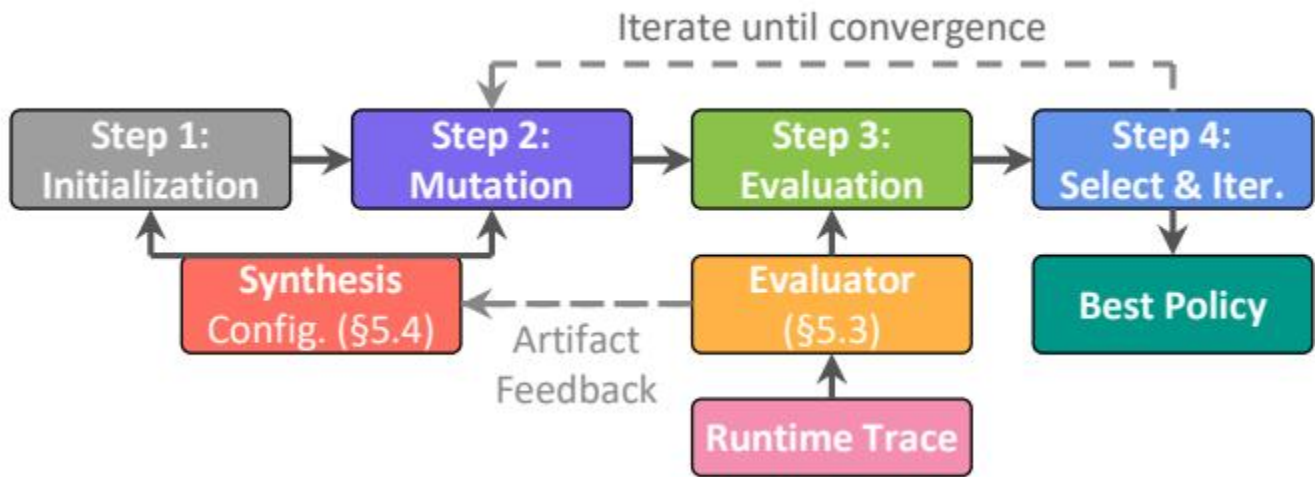
负责处理线上用户请求，执行当前生效的服务策略，连续生成/更新服务计划。同时会定期拍摄运行时快照（滑动窗口内的最近负载、集群状态数据），通过后台线程传输给控制面，不会影响线上请求处理。

控制面（策略演化层）

异步运行，不会中断数据面的服务。收到运行时快照后，启动LLM驱动的程序合成工作流，演化适配当前场景的最优服务策略，演化完成后通过异步热交换机制将新策略部署到数据面，数据面在下一个监控周期自动切换到新策略，无服务中断。

4.2.3. LLM驱动的程序合成 workflow

控制面的核心是LLM驱动的程序合成 workflow，流程如下图（原文Figure5）所示：



该图像是一个示意图，展示了自我演化系统 Autopoiesis 的工作流程，包括初始化、变异、评估和选择迭代四个步骤，直至收敛。该流程强调了在运行时追踪与反馈下持续优化服务策略。相关配置见 § 5.4，评估见 § 5.3。

整个流程分为四个步骤，迭代执行直到收敛或计算预算耗尽：

步骤1：初始化

初始种群包含多个种子策略，每个种子是一套完整的should_reschedule和schedule函数实现，包含不同的算法思路（如贪心调度+固定间隔重调度、ILP优化调度+阈值触发重调度等），为LLM提供多样化的设计模式基础。

步骤2：变异

LLM基于结构化反馈，对当前种群中的高表现候选策略进行代码修改，生成新的候选策略。变异过程会结合两类指导信息：

- 静态权衡知识：系统提示中包含执行模型、权衡关系的描述，让LLM理解代码修改对各个开销项的影响；
- 动态反馈：上一轮评估得到的成本结构 breakdown（如stale开销过高、reconfig开销过高），引导LLM针对性优化。

步骤3：评估与反馈

用轻量服务模拟器重放运行时快照，执行每个候选策略，得到两个输出：

1. 适应度分数：即 T_{total} ，越小表示策略性能越好；
2. 结构化反馈：包含重调度次数 N 、各个开销项的总和（ $\sum t_{stale}$ 、 $\sum t_{reconfig}$ 、 $\sum t_{serve}$ ），用于指导下一轮变异。评估的反馈例子如下（原文Table1）：

Candidate	N	Σt_{stale}	$\Sigma t_{reconfig}$	Σt_{serve}	Ttotal
父策略	2	7.6s	12.0s	25.2s	44.8s
子策略A	5	5.5s	4.0s	24.0s	33.5s
子策略B	1	10.0s	17.0s	23.0s	50.0s

从反馈可以看出，子策略A采用更频繁的轻量重调度，显著降低了stale和reconfig开销，整体性能更优，LLM可以基于该结论引导后续变异方向。

步骤4：选择与迭代

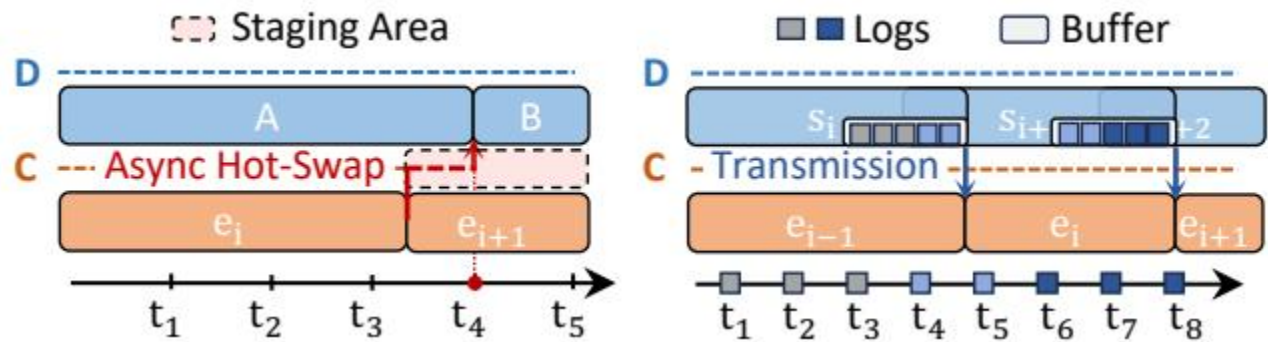
采用MAP-Elites启发的选择机制，保留高表现的候选策略，同时维持种群的多样性，避免陷入局部最优。重复步骤2-4直到演化收敛。

4.2.4. 实现优化

为了保证演化效率和线上可用性，系统实现了四个关键优化：

1. 热启动重演化：下一轮演化以上一轮的高表现候选作为初始种群，不需要从零开始，最多减少83%的演化时间；
2. 多级超时机制：设置演化周期超时、候选策略执行超时，避免错误的候选策略（如死循环、调度时间过长）卡住演化流程；
3. 异步策略热交换：策略是纯函数，替换时只需要替换代码引用，开销可以忽略，不会中断线上服务；

4. **滑动窗口快照**：数据面后台线程异步收集运行时数据，不会占用请求处理路径的资源。热交换和快照的流程如下图（原文Figure6）所示：



该图像是示意图，展示了异步热交换和传输过程中的数据流与缓冲区的状态变化。左侧部分显示了作业 A 和 B 在时间 t_4 的异步热交换；右侧部分则展现了在时间 t_5 传输过程中日志与缓冲区的交互情况。

5. 实验设置

5.1. 数据集

实验采用三类真实数据集覆盖不同动态场景：

- 1. **ShareGPT**：真实用户对话请求数据集，包含不同长度的prompt和生成结果，用于测试工作负载波动场景；
- 2. **LongBench**：长上下文测试数据集，包含长prompt请求，用于测试长文本场景的性能；
- 3. **Microsoft Azure Functions (MAF) 数据集**：微软Azure云的真实函数调用 trace，包含spot实例的抢占、补充记录，用于测试弹性集群动态场景；
- 4. **瑞士AI中心生产trace**：真实生产环境的LLM服务负载trace，包含稳定和波动两种负载模式。

5.2. 评估指标

论文采用两个核心评估指标，以下是详细说明：

5.2.1. Trace完成时间（Trace Completion Time, TCT）

概念定义

指处理完给定trace中所有请求的总端到端时间，包含调度开销、重配置开销和服务时间，越小表示系统性能越好，是本文的核心优化目标。

数学公式

即问题定义中的 T_{total} ：
$$\mathrm{T}_{\{\mathrm{total}\}} = \mathrm{t}_{\{\mathrm{scheduled}\}}(1) + \mathrm{t}_{\{\mathrm{serve}\}}(1) + \sum_{\mathrm{i}=2}^{\mathrm{N}} \Big[\mathrm{t}_{\{\mathrm{stale}\}}(\mathrm{i}) + \mathrm{t}_{\{\mathrm{reconfig}\}}(\mathrm{i}) + \mathrm{t}_{\{\mathrm{serve}\}}(\mathrm{i}) \Big]$$

符号解释

与问题定义中的符号完全一致。

5.2.2. 吞吐量（Throughput）

概念定义

指单位时间内系统处理的token数量，越大表示系统的处理能力越强，是衡量LLM服务性能的核心指标。

数学公式

$$\text{Throughput} = \frac{\text{Total processed tokens}}{\text{Trace Completion Time}}$$

符号解释

- Total processed tokens：trace中所有请求的输入+输出token总数；
- Trace Completion Time：处理完所有请求的总时间。

5.3. 对比基线

论文选择了三个对应场景的SOTA LLM服务系统作为基线：

1. **DistServe**：面向同构集群、工作负载波动场景优化的SOTA系统，作为同构集群实验的基线；
2. **HexGen**：面向异构GPU集群优化的SOTA系统，作为异构集群实验的基线；
3. **SpotServe**：面向弹性云集群、spot实例场景优化的SOTA系统，作为弹性集群实验的基线。

5.4. 实验环境

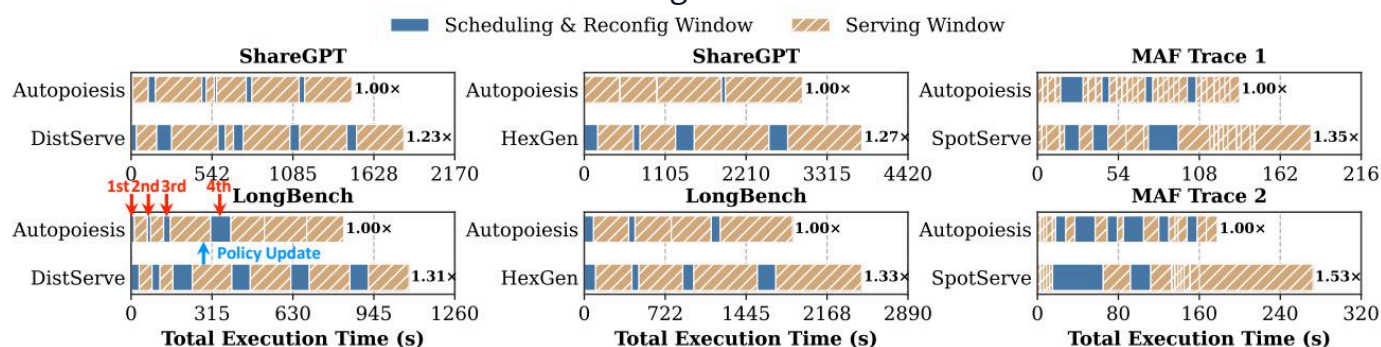
实验采用三类集群配置：

1. **同构集群**: 32张NVIDIA H100-80G GPU, 节点内NVLink带宽300GB/s, 节点间InfiniBand带宽50GB/s, 主要动态是工作负载波动;
2. **异构集群**: 64张不同型号GPU (20张A100-40G、20张A100-80G、8张H100-80G、16张H20-96G), 节点间带宽20Gb/s, 主要动态是工作负载波动+硬件异构性;
3. **弹性云集群**: 最多64张A100-80G GPU, GPU数量随spot实例抢占/补充在24-64之间波动, 主要动态是集群规模变化。程序合成的LLM采用GLM-4.5-Flash, 框架本身是模型无关的, 可以替换为任意代码生成能力的LLM。

6. 实验结果与分析

6.1. 核心端到端结果

三类场景的端到端性能对比结果如下图 (原文Figure7) 所示:



该图像是一个比较不同系统性能的示意图, 展示了在不同情况下, Autopoiesis、DistServe、HexGen、SpotServe的调度与重配置窗口和服务窗口的总执行时间。图中显示了各系统在ShareGPT、LongBench和MAF Trace中的表现。

6.1.1. 同构集群 (vs DistServe)

Autopoiesis相比DistServe最高降低31%的总执行时间, 平均降低27%:

- 负载波动剧烈的阶段: Autopoiesis演化出频繁轻量重调度的策略, 降低65%的调度和重配置开销, 快速响应负载变化, 仅服务 latency 微增2%, 整体端到端时间降低25%;
- 负载稳定的阶段: Autopoiesis演化出低频率、深度调度的策略, 多投入10%的调度和重配置开销生成更优的计划, 服务 latency 降低22%, 整体端到端时间降低11%。

6.1.2. 异构集群 (vs HexGen)

Autopoiesis相比HexGen最高降低33%的总执行时间, 平均降低30%。HexGen由于异构场景的调度空间爆炸, 单次调度需要300s以上, 而Autopoiesis演化出了异构场景专属的调度规则 (如限制

张量并行在节点内、为不同模型匹配最优GPU类型)，调度时间最多降低92.2%，同时保持接近最优的服务效率。

6.1.3. 弹性云集群（vs SpotServe）

Autopoiesis相比SpotServe最高降低53%的总执行时间，平均降低44%：

- 集群稳定阶段：演化出激进重配置的策略，接受较高的重配置开销，换取更低的服务 latency；
- 实例波动频繁阶段：演化出保守重配置的策略，只做部分调整，降低重配置开销，仅牺牲少量服务效率。而SpotServe采用固定的重配置策略，无法适配这两种场景的变化，性能远低于Autopoiesis。

6.2. 场景化案例分析

论文针对三类典型场景做了详细的策略对比分析，结果如下：

6.2.1. 工作负载波动场景

对比了演化策略与两个极端基线（贪心策略：轻量调度+频繁重调度；ILP策略：深度调度+极少重调度）的性能，结果如下（原文Table2）：

Trace	Policy	N	Σ tstale	Σ treconfig	Σ tserve	Over
Volatile（波动）	Greedy	10	65.1s	244.8s	422.1s	95,72
	ILP	1	104.7s	0.0s	686.5s	88,55
	Ours	9	17.5s	217.6s	416.3s	107,5
Stable（稳定）	Greedy	10	27.2s	38.0s	120.5s	436,3
	ILP	1	34.3s	0.0s	113.3s	548,8
	Ours	2	4.3s	4.2s	115.8s	651,2

可以看到：

- 波动场景下，贪心策略优于ILP策略，而演化策略结合了两者的优势，频繁重调度保持计划新鲜，同时优化调度逻辑降低stale开销，吞吐量比最优基线高12%；
- 稳定场景下，ILP策略优于贪心策略，演化策略只进行2次重调度，同时优化调度逻辑降低调度开销，吞吐量比最优基线高19%。

6.2.2. 弹性集群动态场景

对比演化策略与两个极端基线（全迁移策略：每次都重配置到全局最优计划；最小迁移策略：每次只做必要的最小调整）的性能，结果如下（原文Table3）：

Trace	Policy	Σ tstable	Σ treconfig	Overall Thpt
Volatile（波动）	Full-migration	9.7s	108.8s	8,433 t/s
	Minimal-migration	1.6s	8.4s	13,304 t/s
	Ours	0.7s	44.4s	14,843 t/s
Stable（稳定）	Full-migration	9.6s	67.9s	11,080 t/s
	Minimal-migration	2.1s	6.3s	10,187 t/s
	Ours	0.6s	30.8s	12,731 t/s

演化策略在两种场景下都取得了最高吞吐量，相比最优基线分别提升16%和15%，原因是演化出了自适应的重配置策略：波动场景下降低重配置激进程度减少开销，稳定场景下适度重配置提升服务效率。

6.2.3. 智能体请求调度场景

针对多轮智能体请求调度场景，对比演化策略与两个极端基线（贪心策略：快速调度但计划质量低；MILP策略：最优计划但调度开销大）的性能，结果如下（原文Table4）：

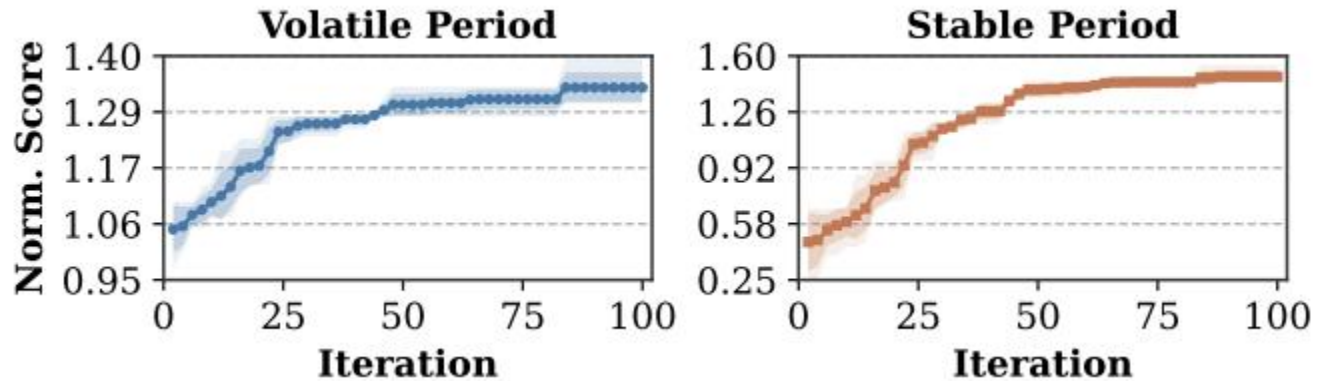
Policy	Trace 1			Trace 2		
	Σ tsched	Σ tserve	Ttotal	Σ tsched	Σ tserve	Ttotal
Greedy	0.1s	10.4s	10.5s	0.1s	18.0s	18.1s
MILP	6.9s	25.0s	31.9s	2.2s	10.3s	12.5s
Ours	1.3s	5.6s	6.8s	0.4s	6.7s	7.1s

演化策略在两个trace上都大幅超过基线，相比最优基线分别降低35%和43%的总时间，证明了合成范式可以适配不同的服务场景。

6.3. 消融实验与参数分析

6.3.1. 演化收敛性

不同负载场景下的演化收敛性如下图（原文Figure8）所示：



该图像是一个示意图，展示了在波动期和稳定期的迭代过程中，标准化得分的变化情况。左侧为波动期的得分曲线，右侧为稳定期的得分曲线。两者的得分随着迭代次数的增加而逐渐上升，表明系统在不同动态下的性能表现。

- 负载波动场景：约80轮迭代收敛，耗时不到10分钟；
- 负载稳定场景：约60轮迭代收敛，耗时不到8分钟。演化的评估阶段是可并行的，用10个并行评估线程可以最多加速8倍，进一步降低演化时间。

6.3.2. 热启动的效果

热启动重演化相比冷启动最多减少83%的演化时间，平均减少79%，只有当运行时环境发生剧烈变化时，热启动的收益才会消失。这证明了连续演化的合理性，大部分场景下可以基于上一轮的策略快速迭代优化。

7. 总结与思考

7.1. 结论总结

本文提出了Autopoiesis，一种自演化的LLM服务范式，首次将LLM驱动的在线程序合成引入LLM服务领域，把静态人工设计的服务策略变成了随运行时动态演化的「活代码」：

1. 双平面架构解耦策略执行与演化，保证线上服务不中断；
2. LLM驱动的程序合成工作流自动探索多维度权衡空间，生成适配当前场景的最优策略；
3. 全场景实验验证了性能优势，相比SOTA系统最高提升53%，平均提升34%。该范式打破了传统系统静态策略的限制，为动态系统的策略优化提供了新的思路。

7.2. 局限性与未来工作

论文中指出的自身局限性和未来研究方向包括：

1. **LLM依赖问题**：演化依赖LLM API的稳定性，商用API的成本较高，未来可以用开源的专属代码生成模型降低成本；
2. **LLM幻觉问题**：LLM可能生成语法正确但逻辑错误的策略，虽然现有超时、评估机制可以过滤大部分坏策略，但仍需要更完善的语义校验机制降低幻觉影响；
3. **快照窗口配置**：当前快照窗口大小是固定配置，太长会稀释当前场景的信号，太短会过度拟合临时波动，未来可以设计自适应窗口大小的机制，根据负载 volatility 动态调整；
4. **演化开销**：当前演化周期需要数分钟，未来可以通过 speculative 评估、早停机制进一步降低演化时间；
5. **多目标支持**：当前优化目标是总完成时间，未来可以扩展支持P95延迟、SLO达成率等更多服务目标。

7.3. 个人启发与批判

启发

本文的范式具有很强的迁移性，除了LLM服务，还可以应用到很多动态系统的策略优化场景：

- 云计算的通用资源调度：动态优化虚拟机、容器的调度策略，适配负载波动；
- 数据库查询优化：动态生成适配当前负载的查询优化策略；
- 网络流量调度：动态调整路由、拥塞控制策略，适配网络流量波动。这类场景的共同特点是策略的最优平衡点随运行时动态变化，人工很难设计覆盖所有场景的静态规则，自演化范式可以很好地解决这类问题。

潜在改进点

1. 可以引入多目标优化机制，同时优化吞吐量、延迟、成本等多个目标，适配不同业务的需求；
2. 可以加入安全校验机制，防止演化出违反业务约束（如多租户隔离、数据安全）的策略；
3. 可以设计策略的可解释性模块，方便开发人员理解演化出的策略的逻辑，提升系统的可运维性。整体而言，Autopoiesis代表了系统设计的一个新方向：从人工设计静态规则，转向系统在线自动演化适配环境的动态规则，未来可能会广泛应用到各类复杂动态系统的设计中。